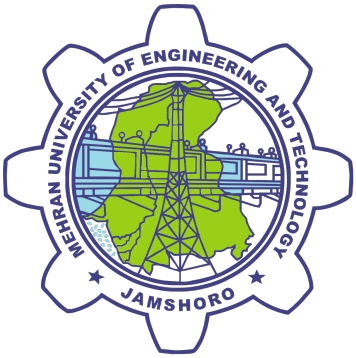
SOFTWARE AS A SERVICE(SAAS) FOR ENCRYPTED FILE STORAGE



A thesis submitted by

Muhammad Yasir (GL) (18SW36)

Nusrat Ali (18SW69)

Babar ali memon (18SW30)

Supervisor

Dr. Qasim ali Arain

In the Partial Fulfillment of the Requirements for the Degree of Bachelor of Engineering in Software Engineering

DEPARTMENT OF SOFTWARE ENGINEERING

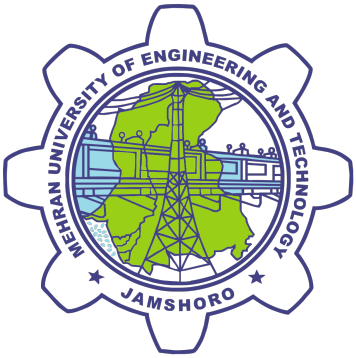
MEHRAN UNIVERSITY OF ENGINEERING & TECHNOLOGY, JAMSHORO

October, 2022

DEDICATION

This humble effort is
DEDICATED
to our **PARENTS** and
TEACHERS with
GRATITUDE and RESPECT

DEPARTMENT OF SOFTWARE ENGINEERING



CERTIFICATE OF APPROVAL

This is to certify that Project/Thesis report on **"Software as a Service(SAAS) for Encrypted File Storage"** is submitted in partial fulfillment of the requirements for a Bachelor's degree in Software Engineering by the following students:

Muhammad Yasir (GL) (18SW36)

Nusrat Ali (18SW69)

Babar ali memon (18SW30)

Project/Thesis Supervisor	Chairman
Dr.Qasim Ali Arain	Dr. Naeem Mahoto

Dated: _____

ACKNOWLEDGEMENT

First and foremost, we gratefully thank Almighty Allah for providing us with the power and capability to do this tough endeavor. Our families' prayers also assisted us in completing this project, for which we are grateful. We are grateful to our supervisor, Dr. Qasim Ali Arian, for assisting us in completing our final year challenge during the course. We were able to meet the challenge's objectives thanks to their guidance, assistance, and inspiration

CONTENTS

LIST OF TABLES

LIST OF FIGURES

ABSTRACT

Cloud storage specifies how digital data gets stored and retrieved across multiple servers in possibly different locations and managed by a cloud service provider e.g. AWS, azure, etc. cloud storage uses a logical memory model that allows providers to store your data on multiple servers in different locations in a way transparent to you. at the very minimum, cloud storage applications include space for some amount of data and a simple user interface to manage files in the storage. usually, this involves creating a special folder on your computer and monitoring file activity in it. dropbox belongs to this category of cloud storage. more elaborate cloud storage includes sophisticated saas that seamlessly integrates with backend cloud storage. google docs offers this kind of service for google drive.. This application is a cloud-based storage application that provides the services of storing small files with 2 encryption algorithms to make them safe on a remote server and develop the Minimum Viable Product(MVP) with core features and a free subscription.

CHAPTER 1

INTRODUCTION

1.1 OVERVIEW OF SOFTWARE AS A SERVICE

As we know, storing data on our local devices in todays world is very messy as there is a lot of data to store and there are also many security problems that can occur in this rapidly developing distributed system world, so it is very hard to secure sensitive data in todays world. So our application is all about to handle these types of problems, which is a web-based cloud storage application that ensures users they can store and access every type of data from anywhere in the world just through any device and an internet connection At any time, they have to pay only for the services/storage they are consuming.

In order to provide a quicker innovation, adaptable resources, and scale economies, Distributing computer services over the Internet (the "cloud"), such as servers, storage, databases, networking, software, analytics, and intelligence, is known as cloud computing. Because you typically only pay for the cloud services you actually use, you may scale up or down as your business's needs change while

also reducing operational costs and improving infrastructure management.

Software as a Service, or SaaS, is a type of cloud computing. Under this system of software delivery and licencing, software is accessed online via a subscription rather than being purchased and installed on specific computers. but we will provide you with a system in which one can store data to a small extent. Accessed online via a free subscription, and that stored data will be encrypted using two encryption algorithms.

Since the SaaS business model offers so many benefits to both consumers and developers, 33

1.2 PROBLEM STATEMENT

It is a service in which we can store data, and that system can only be accessed online via subscription. Without any subscription, no one can access it or store the data on that system. But other than that, there is one more problem with security. Most people store sensitive data on that system, which can be easily hacked by an intruder. If our SaaS application is encrypted without or with a single encryption algorithm, it can be easily hacked.

1.3 PROPOSED SOLUTION

We are providing you with a system that can be accessed online via a free subscription. in which you can store data to a small extent free of charge. The system will be much more secure because the stored data will be passed through two encryption algorithms.

1.4 FEATURES OF SOFTWARE AS A SERVICE FOR ENCRYPTED FILE STORAGE

The Application will allow users to store small files which will be encrypted twice , the user can upload new files, download stored files, choose service provider and a lot more. Addition the application will have the following features:

1.4.1 Integrity

The data stored on the remote server via the SaaS app will have integrity as it is going to be encrypted twice, so the data cannot be changed while it is stored by any third party or by the service providers themselves.

1.4.2 Availability

Since the application will be deployed on a cloud service and the files stored will also be on the remote servers the system will highly available to users at practically all times.

1.4.3 Confidentiality

The user's confidentiality is ensured as the files are being encrypted while being persisted to the remote server hence, no one else can access the data which make the system quite confidential.

1.4.4 Scalability

The system will have the ability to scale on demand and as the need for user's storage increases.

1.5 AIM AND OBJECTIVES

To create a SaaS(Software as a Service) application which provide user the ability to log in more securely with a USB key and Allow them to upload and retrieve data to a cloud storage which is encrypted.

1.5.1 Objectives

- Creating a way to allow the user to login with a USB Flash Drive.
- Encrypting the usb key code with 2 encryption methods.
- Allow the user to upload data and then encrypt it twice.
- Have the service decrypt that data while it is being retrieved.
- Allow the user to choose the cloud provider.

1.6 SCOPE/MOTIVATION

In the era of technology everyone wants to save resources, the scope of the project is extended from a day to day user to even an enterprise. The scope of growth for cloud computing is immense. More organizations should prioritize the use of cloud technology. When data gets stored in the cloud, it becomes easier to ensure performance, integrity, security, and functionality. One limitation would be speed of the network, which controls the speed at which data is managed and processed. If the network is fast enough, everything else about the use of cloud computing will fall in place. Our application will be only an MVP (Minimum Viable Product) initially but as the product succeeds in the market we will start rolling out the

full version with an extensive lot of features.

CHAPTER 2

LITERATURE REVIEW

We conducted a literature review and explored several tools and analysis approaches before beginning to implement our project.

2.1 RELAVENT WORK

2.1.1 Security-aware SaaS placement using swarm intelligence

In order to meet resource shortages and provide a variety of on-demand services, the cloud computing paradigm has developed as a new, powerful service delivery method (eg, software, storage, network, etc.). One of the most well-liked service models is software as a service (SaaS). SaaS can be provided in a composite form to satisfy consumers' rising demands. While this method has some benefits, such as flexibility and reuse, it also poses the issue of how to manage composite SaaS in a distributed and highly dynamic cloud context. In this essay, we examine the SaaS placement challenge, one of the main problems with SaaS resource management. In this work, we add security issues in SaaS placement strategy because previous attempts have only looked at the SaaS placement problem from the standpoint of resource consumption to maximise SaaS performance

and decrease resource usage. In actuality, one of the key elements affecting the effectiveness of the composite SaaS is security risk. To suggest a placement approach for SaaS that is security conscious, we use a multi-swarm form of particle swarm optimization. Additionally, the placement algorithm has been hybridised with a cooperative learning technique to improve the efficiency with which the best candidate servers' information is used to produce placement plans. Studies reveal that our solution performs better than the methods currently used for SaaS placement.

2.1.2 Auto encryption algorithm for uploading data on cloud storage

A scalable, dependable, and quickly developing technology is cloud computing. It provides a variety of pay-per-use services to its users. These include things like processing, storage, and diverse applications, among other things. With its extensive and reliable infrastructure, it is present all over the world. The storage security of cloud computing is the main topic of this study. In recent years, it has been discovered that cloud storage is not entirely safe. Regarding user data security and privacy, it is still in its infancy. Numerous algorithms have been developed and put into use to date; nevertheless, all of them rely on the user supplying a key rather than producing

the key themselves to encrypt data. This research study offered an algorithm that will generate a key based on the data supplied and then encrypt data using the generated key. Data that has been encrypted will be uploaded to a cloud storage service, and the key will be safely stored on a local server for later decryption.

2.1.3 A Business Model for Cloud Computing Based on a Separate Encryption and Decryption Service

Businesses typically keep their data in internal storage and create firewalls to prevent unauthorised access. Additionally, they standardise data access protocols to stop insiders from leaking information without authorization. Data will be kept in storage offered by service providers in cloud computing. Data protection must be a practical option for service providers, particularly to guard against unauthorised insiders disclosing customer information. An effective way to preserve the privacy of your information is to save the data in encrypted form. The system administrators may simultaneously get encrypted data and decryption keys if a cloud system is in charge of both data storage and encryption/decryption duties. This puts a danger on information privacy because it enables them to access information without permission. This paper suggests a cloud computing business model based on the idea of separating the storage

service from the encryption and decryption service. Additionally, the person in charge of the data storage system must avoid storing information in plaintext, and the person in charge of data encryption and decryption must remove all data after the encryption or decryption calculation is finished. In this paper, the proposed business model is described using the example of a CRM (Customer Relationship Management) service. The excellent service makes use of three cloud-based technologies, including a storage system, a CRM application system, and a system for encryption and decryption. According to the main idea of the suggested business model, one service provider runs the encryption and decryption system while other providers run the storage and application systems. Additionally, this paper makes recommendations for a multi-party Service-Level Agreement (SLA) that can be used with the suggested business model.

2.1.4 Enhanced Security for Cloud Storage using File Encryption

The phrase "cloud computing" refers to a network that provides extraordinary processing power, a broad range of storage options, and unbelievable calculation speeds. Individual customers, business organisations, and social media platforms are all migrating to the wonderful world of cloud computing. On the other hand, with cloud

storage come security concerns about data availability, data integrity, and confidentiality. Authentication and authorisation for data access are more than necessary because the cloud is only a collection of physically present supercomputers dispersed around the globe. Our efforts aim to mitigate these security risks. The suggested practise recommends encrypting the data before uploading them to the cloud. In addition to encrypting the user-uploaded data, which is done to preserve its integrity and secrecy, the data is also made accessible only after a successful authentication.

2.1.5 Offering security diagnosis as a service for cloud SaaS applications

Web services are becoming essential parts of Software as a Service (SaaS) applications in cloud ecosystem contexts due to the maturation of service-oriented architecture (SOA), microservices architecture, and Web technologies. These technologies, while their obvious benefits, expose SaaS applications to both new and common attack vectors. The absence of security enforcement and loss of control over sensitive data that SaaS apps modify increases this security risk. To assess the security posture of SaaS apps and find possible information flow vulnerabilities, we deliver our solution as Security Diagnosis as a Service (SDaaS). We assess our framework's perfor-

mance, scalability, and detection accuracy. The tests are carried out on benchmark apps to evaluate vulnerability detection services and technologies. We compare other tools, such as static code analyzers, penetration testers, and anomaly detectors, to our solution. The results of the trials demonstrate that the framework is a workable defence against breaches of data integrity and confidentiality. The evaluation's findings show that SDaaS identifies information flow vulnerabilities with high accuracy, performance, and scalability as well as a small resource footprint.

CHAPTER 3

DESIGN AND ARCHITECTURE

This chapter covers the methodology we have used in our project. In this chapter we have discussed about the architecture design of our project.

3.1 METHODOLOGY:

Before developing any application, we have to choose the methodology which we should follow to develop that application. There are many methodologies which can be used as application methodology. For our application SaaS for encrypted file storage, we have used Waterfall model. This methodology has some stages. In first stage we collect the requirements of our application. After gathering requirements, we check their analysis and feasibility. The second stage is design in which we convert requirements into system design. After designing stage, the next one is implementation or coding in which we develop the application using above requirements and system design by implementing the features in any programming languages. After implementation we test and verify the implemented code and check that either it is working according to the requirements and

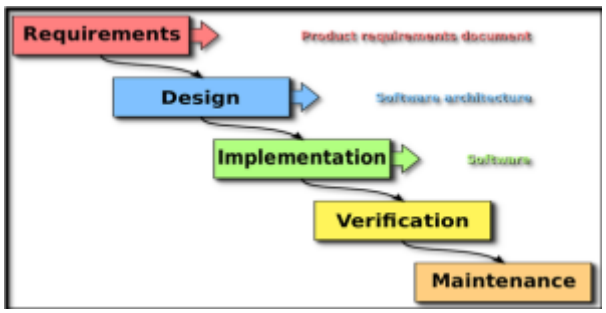


Figure 3.1: waterfall model

design or not. After testing successfully, we deploy that application and work on its maintenance.

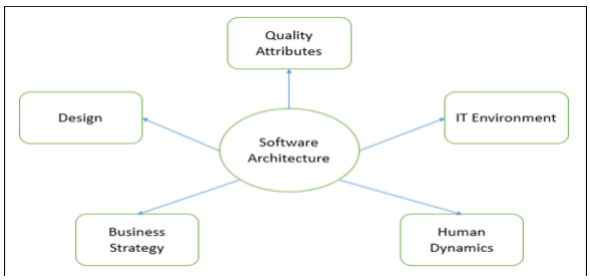


Figure 3.2: System Architecture

3.2 DESIGN

3.2.1 ARCHITECTURE

Architecture of any application describes the conceptual design and behavior of the system. The architecture of any application defines the major modules and entities of an application their dependencies and interactions among them. It also includes some attributes like quality attributes, business strategies, human dynamics and IT environment etc. The architecture of an application will decide the behavior and flow of system. The relationship among the components will also describes the Architecture of the system. Architecture shows the Blue Print of a system

Architecture comes with the system design in which we will specify the elements of the system. It describes the requirements desired quality, hardware architecture, hardware architecture design their modifications and implementation constraints also.

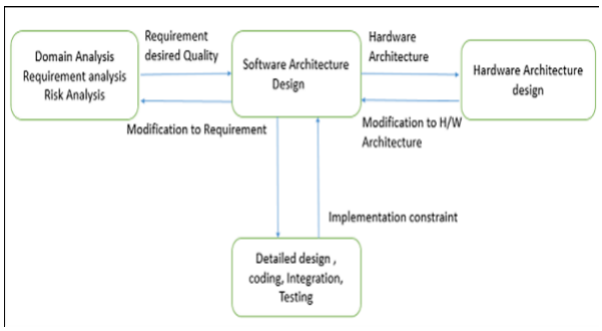


Figure 3.3: System Architecture

It consists of the following components:

3.2.2 Client

The client is software which is usually our web browser. Client is something that send request for resource or service and consumes that service. In area of web applications the browsers that we normally used to access the different web pages are the clients. From browser we send request for particular source or service and use the response of that request.

3.2.3 Server

Server is software that is used to generate the response of 21 client's requests. It receives the client requests, resolve those requests and start processing the requests and generate the responses for them and send those responses back to the clients and let them consume those services.Basically it is a service provider.

3.2.4 Database

Database is collection of an organized data. The database is used to store the data of an application in an arranged manner. In relational database management systems we use to store the data in the form of tables. One table represents the data of one entity of that application. The table contains the data in the form of tuples/rows. One row define one data record for that entity. Server stores data of clients in the database and process their requests and fetch data from database and send data in the response.

3.3 UML DESIGN MODELS

It is abbreviated as Uniform Modeling Language. It is modeling language that is used to describe the structure, behavior of an application. There are different kinds of UML design models that can be designed to describe the structure and behavior of any system application. UML Design models help the system designers in the designing phase of an application.

3.3.1 Flow chart

Flow chart is a type of unified modeling language diagram that is used to depict the process and flow of sequential order of actions

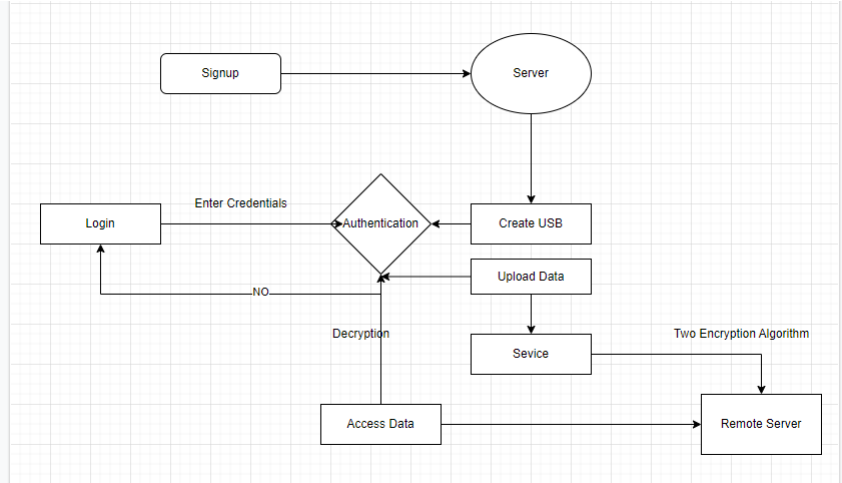


Figure 3.4: flow chart

that would be taken to process something in the application. Basically they are used to create for better understanding of application workflow. There are many online tools that we can use to design the flow charts for our applications.

3.3.2 USE CASE DIAGRAM

Use case diagrams are basically the type of unified modeling language diagrams in which list of event actions are defined which shows the interactivity among the components of an application. In use case diagrams we draw the actors that basically the main assets and users of our application. We also draw the sequence of actions that made by those actors.

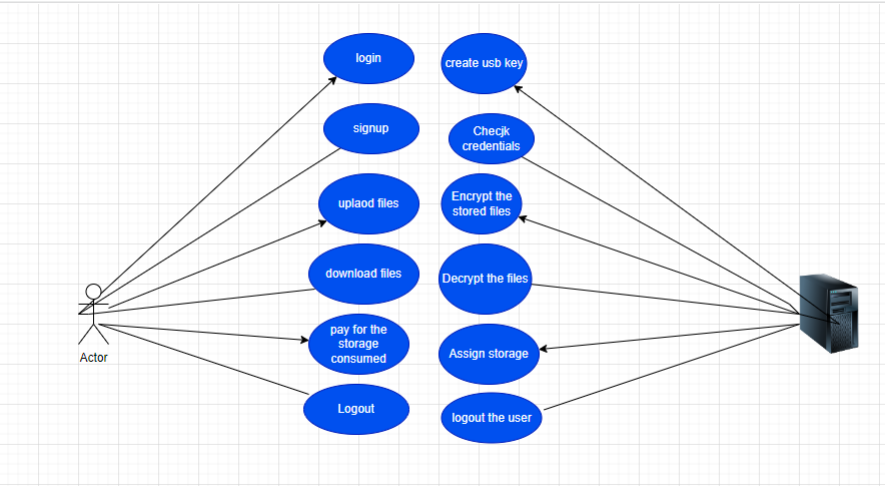


Figure 3.5: Use case diagram

3.3.3 3ERD DIAGRAM

ERD stands for Entity relationship diagram. It is the type of unified modeling language in which we describe all the entities of our application. The relationship between the entities. There are many terms that we used to figure in the entity relationship diagram like normalization, optimicity, multiplicty and other as well. Refer to fig given below:

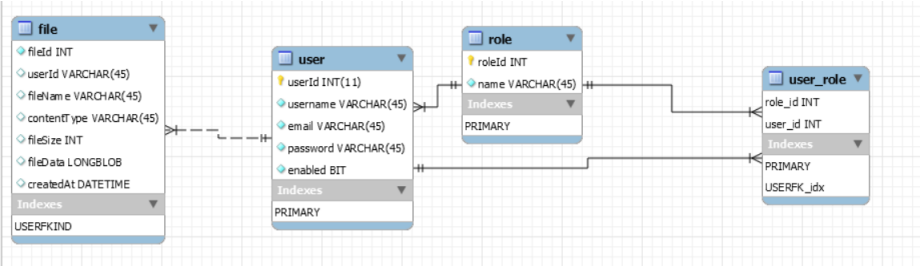


Figure 3.6: Entity relation diagram

3.3.4 CLASS DIAGRAM

This diagram is also a type of uml diagram in which we show the relationships between the classes of an application. This will help us to understand the implementation of an application. Refer to fig given below:

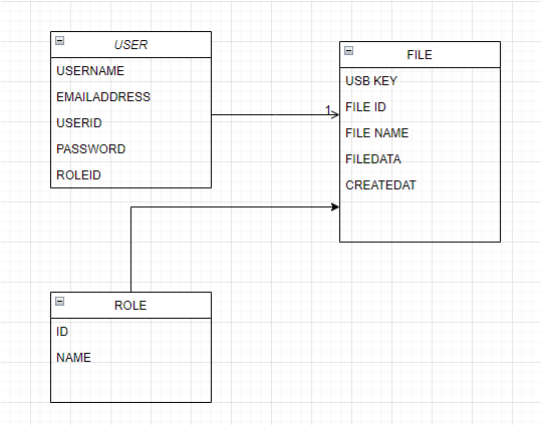


Figure 3.7: class Diagram

CHAPTER 4

TOOLS AND TECHNOLOGY

4.1 FRONT-END TECHNOLOGIES USED

- HTML5
- CSS3
- JAVASCRIPT
- BOOTSTRAP
- JWT
- FILE SYSTEM ACCESS API

4.1.1 HTML

Widely used language that creates HTML pages. HTML stands for Hypertext Markup Language, the latest version of the HTML is 5. HTML was created to de ne the structure of the documents, like, title, heading, paragraph, lists, forms, etc. HTML uses tags to format the documents. HTML is a language, which makes the skeleton of a web page, one cannot make a website or web application without using it.

4.1.2 CSS

HTML provides a basic way of designing a web page. However, when it comes to the designing part, HTML becomes very di cult. CSS (cascading style sheets) can make decorating web sites easily. CSS describes how web pages should look. We can easily separate CSS les from HTML les and use them in the HTML using ¡link¿ tag, so that code is easy to and, modify, and changes the look of your website. CSS has been used in our application for styling purposes and to provide a better look and feel. We have used CSS in our web application for a better look and feel. Using CSS, one can control the color of the text, the style of fonts, the spacing between paragraphs, background images, layout design, etc. CSS also provides responsiveness to open web pages on different devices like smartphones, tablets, etc. It also provides a variety of e effects and animations to make the web page dynamic. In this project, CSS is used to improve the look and feel of webpages.

4.1.3 BOOTSTRAP

Bootstrap is the most popular open-source front end web framework for designing websites and web applications. For easier and faster development. It includes HTML and CSS based design templates

11 for typography, forms, buttons, navigation, modals, image and many other features. We can develop responsive and interactive web pages with bootstrap. Bootstrap is used with HTML tags in form class attribute, whenever we want to use a bootstrap component, we have to reference the bootstrap le through HTML link tag and then use the class, which we want to use. Bootstrap is used in this project to make the webpages more interactive and dynamic.

4.1.4 JAVASCRIPT

JavaScript is a client-side scripting language. It allows inserting the code into your web pages. It has many controls in web applications. 12 It has control to handle the web pages on the client-side. JavaScript is used to entertain the client's request means it takes or carry the requests from the clients and show them the outputs on the screen. The JavaScript is used is on this website for some event handling and some client-side coding.

4.1.5 REACT JS

A JavaScript package called React is used to create user interfaces. It has surpassed jQuery, express, and Angular to become the most widely used front-end development framework, according to Statista. React is special because it enables you to build reusable components

that make it simple to read and manage your code. React immediately modifies a component in response to user interaction, making your app more responsive and quick. React components are by nature short, autonomous pieces of code that are identified by the functionality they provide, such as buttons, menus, or forms. A self-contained module that renders some UI is called a component. React was developed to address problems that arise when creating huge applications with dynamic data , Large-scale application development differs from typical "imperative" programming, where you must

4.1.6 MATERIAL UI

A React UI framework called Material-UI provides a large range of pre-made components that you may utilize in your React project. These components can be used by developers with the Material-UI styling library, which is also accessible in the React universe. The fact that Material-UI offers a huge selection of components that may be used to build practically any form of UI possible is one of its significant advantages. For instance, you can discover both simpler and more complicated components, such as buttons, cards, sliders, dialogues, grids, and tooltips.

4.1.7 JWT

An open standard (RFC 7519) called JSON Web Token (JWT) is used to safely transport data between parties in the form of JSON object, It is small, readable, and digitally signed by the Identity Provider using a private key or a public key pair (IdP). As a result, the token's legitimacy and integrity can be confirmed by other parties , Using JWT serves to assure data authenticity rather than to conceal data. JWT is not encrypted; it is signed and encoded , JWT is a stateless, token-based authentication method. Since the session is client-side based and stateless, the server does not entirely rely on a datastore (database) to store session data.

4.1.8 FILE SYSTEM ACCESS API

Through the usage of the device's file system, this API enables developers to create robust apps that communicate with other (non-Web) programmes on the user's device. IDEs, photo and video editors, text editors, and other prominent apps where customers anticipate similar capabilities are only a few examples. This API enables a web app to read or write modifications to files and folders on the user's device after the user gives access. This API offers the ability to open a directory and enumerate its contents in addition to reading and

writing files. Additionally, web apps can utilise this API to save references to the files and folders they have been granted access to, enabling the web apps to later regain access to the same information without necessitating that the user select the appropriate option.

4.2 BACKEND TECHNOLOGIES

- SPRING BOOT
- SPRING MVC
- SPRING SECURITY

4.2.1 SPRING BOOT

Spring Boot is java framework. It is an open source framework and used to make a micro service. It creates standalone and production ready applications. Java developers take huge advantage in developing standalone and production ready applications. Spring Boot enables the auto configuration in large scale-based applications. It makes easy to understand and develop spring applications. It also increases the productivity and reduce the time of development. There are many other benefits of spring boot framework.

4.2.2 SPRING MVC

Spring MVC is a framework of java programming language that is used to create web applications. It implements Model-View Controller design pattern and separates the business logic layer from database layer. In MVC we create model for transferring data, View for data presentation and inserting data, Controller for making com-
14 munication between model and view. Spring MVC implements all basic features of spring framework like Inversion of control, dependency injection. It provides graceful solution to use MVC framework with the help of Dispatcher Servlet

4.2.3 SPRING SECURITY

Spring Security is java-based framework that is used to create web applications secure. It is highly powerful and customizable authentication and access control framework. It provides authorization and authentications to java-based web applications.

4.3 DATABASE

The database which is used in project development is MySQL. because MySQL provides many features like data security which is very crucial for developing any database, high performance through high-

speed transnational processing.

- MYSQL
- SERVER

4.3.1 MYSQL

Open-source relational database management system. It provides a structured query language SQL. It is fast and easy to use RDBMS that is being used in small and big projects. It works on many programming languages like PHP, C, C++, JAVA, etc. It is used 15 for accessing, adding, and managing the data in a database because its reliability is high and has a quick response. We have designed the database of our website using MYSQL. All queries have been written using MySQL. We have designed the database of our website using MySQL. All queries have been written using MySQL.

4.3.2 SERVER

The server that we have used to create our application is Tomcat local host server. Basically, this Tomcat server is default server of Spring Boot application.

4.4 TOOLS

- INTELLIJ IDEA
- MYSQL WORKBENCH
- POSTMAN
- DRAWS.IO

4.4.1 INTELLIJ IDEA

To increase developer productivity, IntelliJ IDEA is an Integrated Development Environment (IDE) for JVM languages. By offering sophisticated code completion, static code analysis, and refactoring's, it takes care of the tedious and repetitive duties for you and frees you up to concentrate on the positive aspects of software development, making it both efficient and pleasant.

4.4.2 MYSQL WORKBENCH

A unified visual tool for database architects, developers, and DBAs is MySQL Workbench. Data modelling, SQL creation, and extensive administrative tools for server configuration, user management, backup, and other tasks are all provided by MySQL Workbench. There are versions of MySQL Workbench for Windows, Linux, and

Mac OS X.

4.4.3 POSTMAN

Application programming interface (API) development tools like Postman make it easier to create, test, and alter APIs. This tool contains almost all of the features that a developer could possibly need. More than 5 million developers use it each month to make developing APIs quick and straightforward. It can create several HTTP requests (GET, POST, PUT, and PATCH), save environments for later use, and translate the API into code for different languages (like JavaScript, Python).

4.4.4 DRAWS.IO

Online diagramming software diagrams.net (formerly draw.io) is free to use. It functions as a flowchart creator, network diagram software, online UML creator, ER diagram tool, database schema designer, online BPMN builder, circuit diagram maker, and more. Gliffy, Lucidchart, and.vsdx files can all be imported into draw.io.

CHAPTER 5

IMPLEMENTATION

In this chapter we will show you snap shots of the code of our project's front-end and back-end.

5.1 INTERFACE

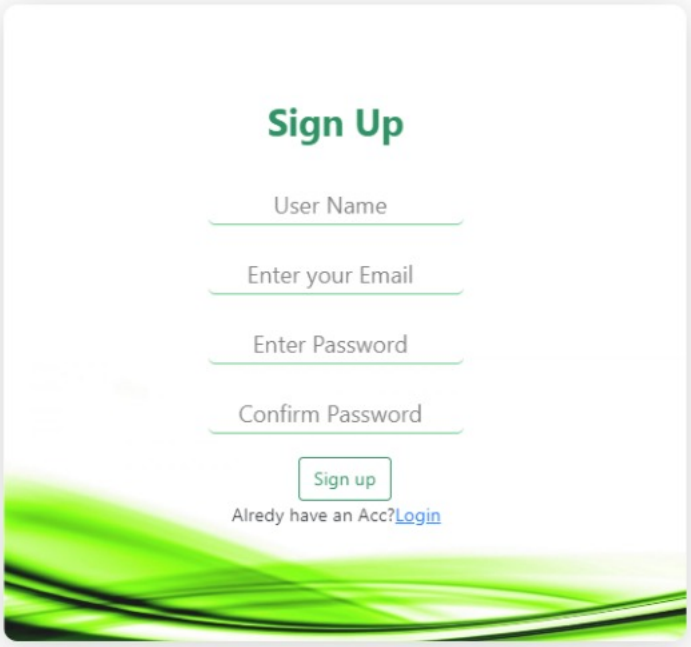


Figure 5.1: signup

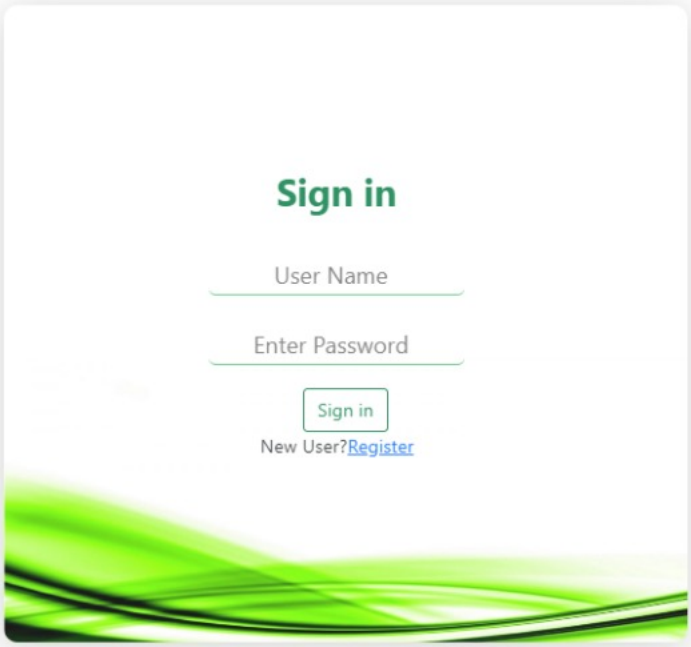


Figure 5.2: signin

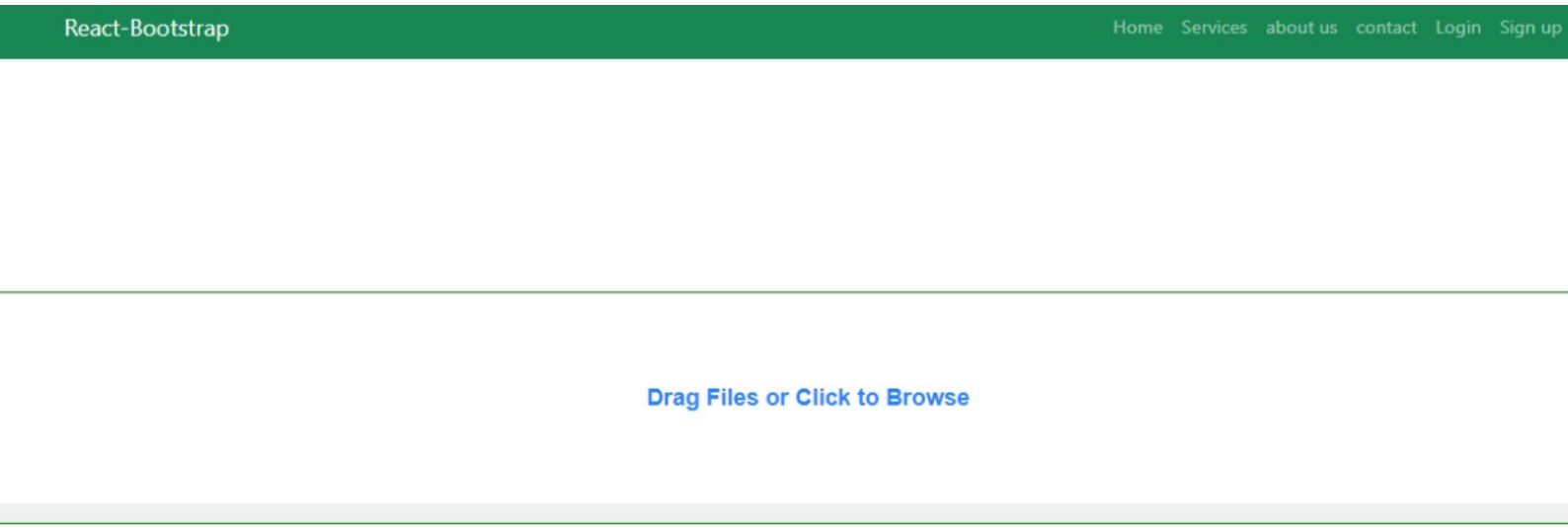


Figure 5.3: upload file

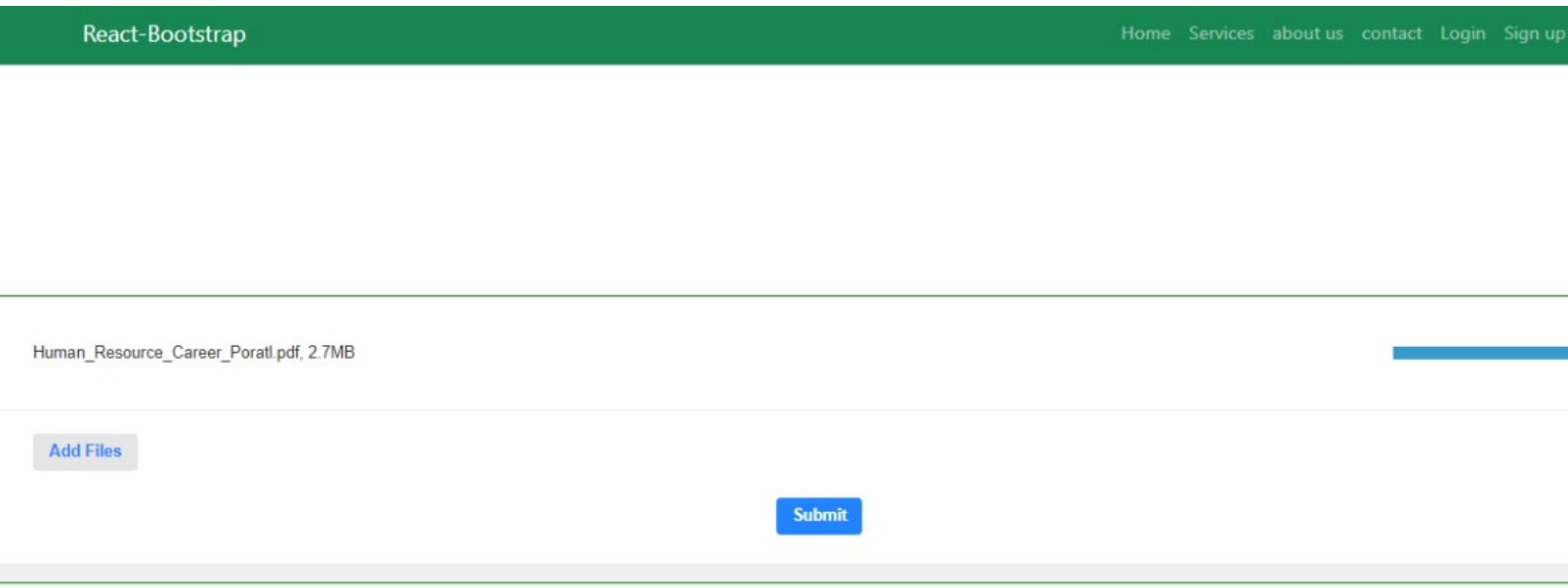


Figure 5.4: Submit

5.2 FRONT-END IMPLEMENTATION

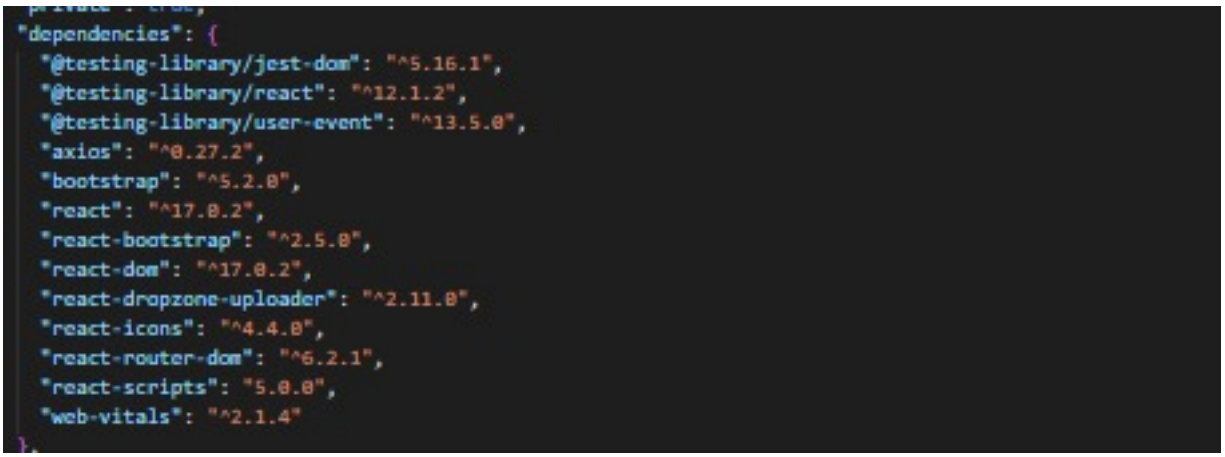


Figure 5.5: Dependencies on which our project depends

components > Register > JS Register.js > [@] Register

```
import Button from "react-bootstrap/Button";

const Register = () => {
  const navigate = useNavigate();

  const goToFileUploader = () => {
    navigate("/Fileuploader");
  };

  const goToLogin = () => {
    navigate("/Login");
  };

  return (
    <>
      <section>
        <div className="main">
          <h2 className="SignUp"> Sign Up </h2>
          <form className="form">
            <div className="inputBox">
              <input type="text" placeholder="User Name" />
            </div>

            <div className="inputBox">
              <input type="email" placeholder="Enter your Email" />
              {/* <input type="text" placeholder="your placeholder" onfocus="this.placeholder=''" onblur="this.placeholder='your placeholder'" /> */}
            </div>
            <div className="inputBox">
              <input type="password" placeholder="Enter Password" />
            </div>

            <div className="inputBox">
              <input type="password" placeholder="Confirm Password" />
            </div>
          </form>
          <div>
            <Button  variant="outline-success" onClick={()=>goToFileUploader()}>Sign up</Button>
            </div>
          <div>

            <div>
              <p>
                Alredy have an Acc?
                <a href="" onClick={()=>goToLogin()}>
                  Login
                </a>
              </p>
            </div>
          </div>
        </div>
      </section>
    </>
  );
};

export default Register;
```

Figure 5.6: Coding for signup page

```jsx
import React from "react";
import { useNavigate } from "react-router-dom";
import "./signin.css";
import { Button } from "react-bootstrap";

const Signin = () => {
  const navigate = useNavigate();
  const goToRegister = () => {
    navigate("/Register");
  };
  return (
    <>
      <section>
        <form >
          <div className="main">
            <h2 className="SignUp success"> Sign in </h2>
            <div className="form">

                <div className="inputBox">
                  <input type="text" placeholder="User Name" />
                </div>

                <div className="inputBox">
                  <input type="password" placeholder="Enter Password" />
                </div>

            </div>
            <div>
              <Button  variant="outline-success">Sign in</Button>
            </div>
            <div>
              <p>
                New User?
                <a href="" onClick={()=>goToRegister()} >
                  Register
                </a>
              </p>
            </div>
          </div>
        </form>
      </section>
    </>
  );
};

export default Signin;
```

Figure 5.7: Coding for sign$_in page$

```
const Fileuploader = () => {
    const getUploadParams = () => {
      return { url: 'https://httpbin.org/post' }
    }

    const handleChangeStatus = ({ meta }, status) => {
      console.log(status, meta)
    }

    const handleSubmit = (files, allFiles) => {
      console.log(files.map(f => f.meta))
      allFiles.forEach(f => f.remove())
    }

    return (
      <>
        <Nav/>
        <Dropzone
          getUploadParams={getUploadParams}
          onChangeStatus={handleChangeStatus}
          onSubmit={handleSubmit}
          styles={{ dropzone: { minHeight: 200, maxHeight: 250, borderColor:"green",marginTop:200 } }}
        />


      </>

    )

  }

  export default Fileuploader;
```

Figure 5.8: Code for uploading file

5.3 BACK-END IMPLEMENTATION

5.3.1 Layout

5.3.2 Folder Structure

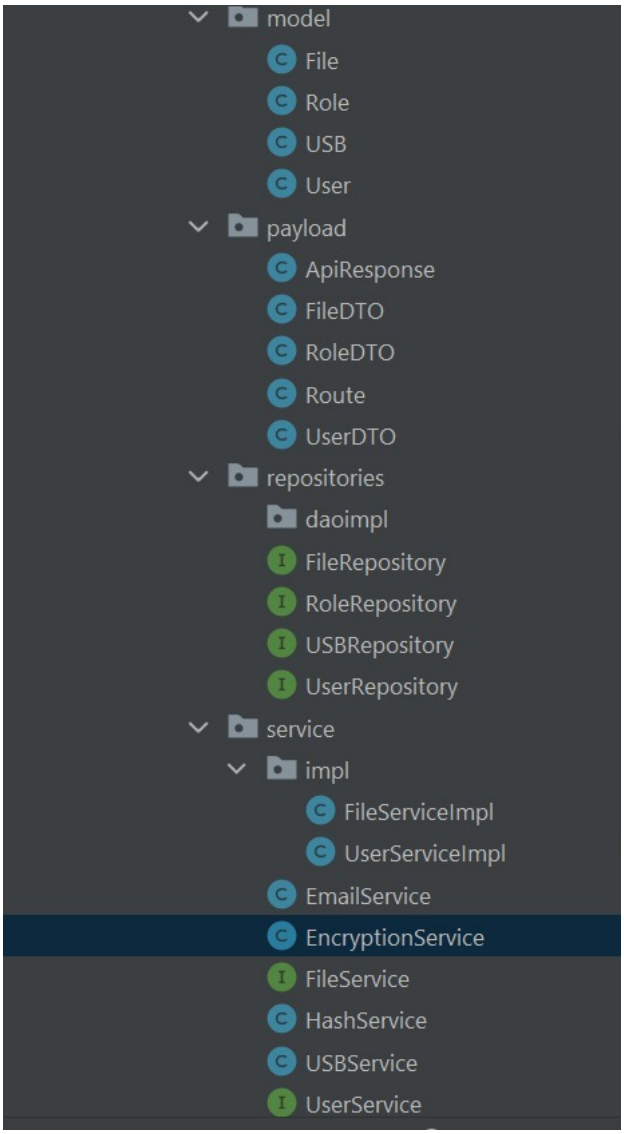


Figure 5.9: Folder structure for backend code

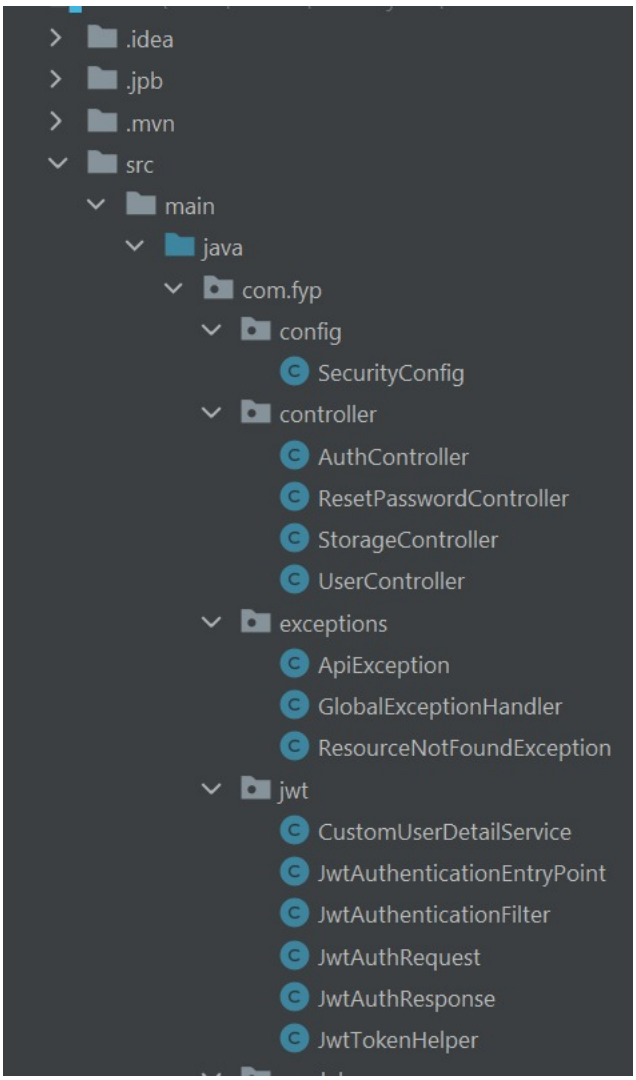


Figure 5.10: Folder structure for backend code

5.3.3 Login/JWT

```
@PostMapping("/login")
public ResponseEntity<JwtAuthResponse> createToken(@RequestBody JwtAuthRequest request)
{
    UserDetails userDetails;
    Map<String,Object> claims = new HashMap<>();
    String token;
    JwtAuthResponse response;

    //load user
    userDetails = this.userDetailsService.loadUserByUsername(request.getUsername());

    UserDTO user = userService.getByUsername(userDetails.getUsername());

    //authenticate request
    authenticate(request.getUsername(), request.getPassword());

    //add claims
    claims.put("userId",user.getId());
    claims.put("routes", AppConstants.ADMIN_ROUTES);
    //generate token
    token = this.jwtTokenHelper.generateToken(claims,userDetails);
    //set and return response
    response = new JwtAuthResponse(token);

    return new ResponseEntity<>(response, HttpStatus.OK);
}
```

Figure 5.11: Authentication and JWT creation

5.3.4 User Service

```java
package com.fyp.service;

import com.fyp.payload.UserDTO;

import java.util.List;

public interface UserService {

    UserDTO saveUser(UserDTO user);
    UserDTO getUserById(Long user_id);
    UserDTO getByUsername(String username);
    UserDTO updateUser(UserDTO user,Long id);
    List<UserDTO> getAllUsers();
    void deleteUser(long id);
}
```

Figure 5.12: User service

5.3.5 Email Service

```
public boolean sendEmail(String subject,String message,String to)
{
    Properties props = System.getProperties();
    String email = props.getProperty("spring.mail.username");
    String password = props.getProperty("spring.mail.password");
    Session session = Session.getInstance(props, getPasswordAuthentication() → {
            return new PasswordAuthentication(email,
                    password);
    });

    session.setDebug(true);

    MimeMessage mime = new MimeMessage(session);
    try{
        mime.setFrom(email);
        mime.addRecipient(Message.RecipientType.TO,new InternetAddress(to));
        mime.setSubject(subject);
        mime.setText(message);
        Transport.send(mime);
        return true;
    }
    catch (Exception e)
    {
        e.printStackTrace();
    }
    return false;
}
```

Figure 5.13: Email Service to send otp for resetting password

5.3.6 Encryption Service

```
package com.fyp.service;

import ...

@Service
public class EncryptionService {
    private Logger logger = LoggerFactory.getLogger(EncryptionService.class);

    private static final int SEED =  16;

    public String encryptValue(String data, String key) {
        byte[] encryptedValue = null;

        try {
            Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
            SecretKey secretKey = new SecretKeySpec(key.getBytes(), s: "AES");
            cipher.init(Cipher.ENCRYPT_MODE, secretKey);
            encryptedValue = cipher.doFinal(data.getBytes( charsetName: "UTF-8"));
        } catch (NoSuchAlgorithmException | NoSuchPaddingException | InvalidKeyException
                | UnsupportedEncodingException | IllegalBlockSizeException | BadPaddingException e) {
            logger.error(e.getMessage());
        }

        return Base64.getEncoder().encodeToString(encryptedValue);
    }
```

Figure 5.14: Method to encrypt files in Encryption Service

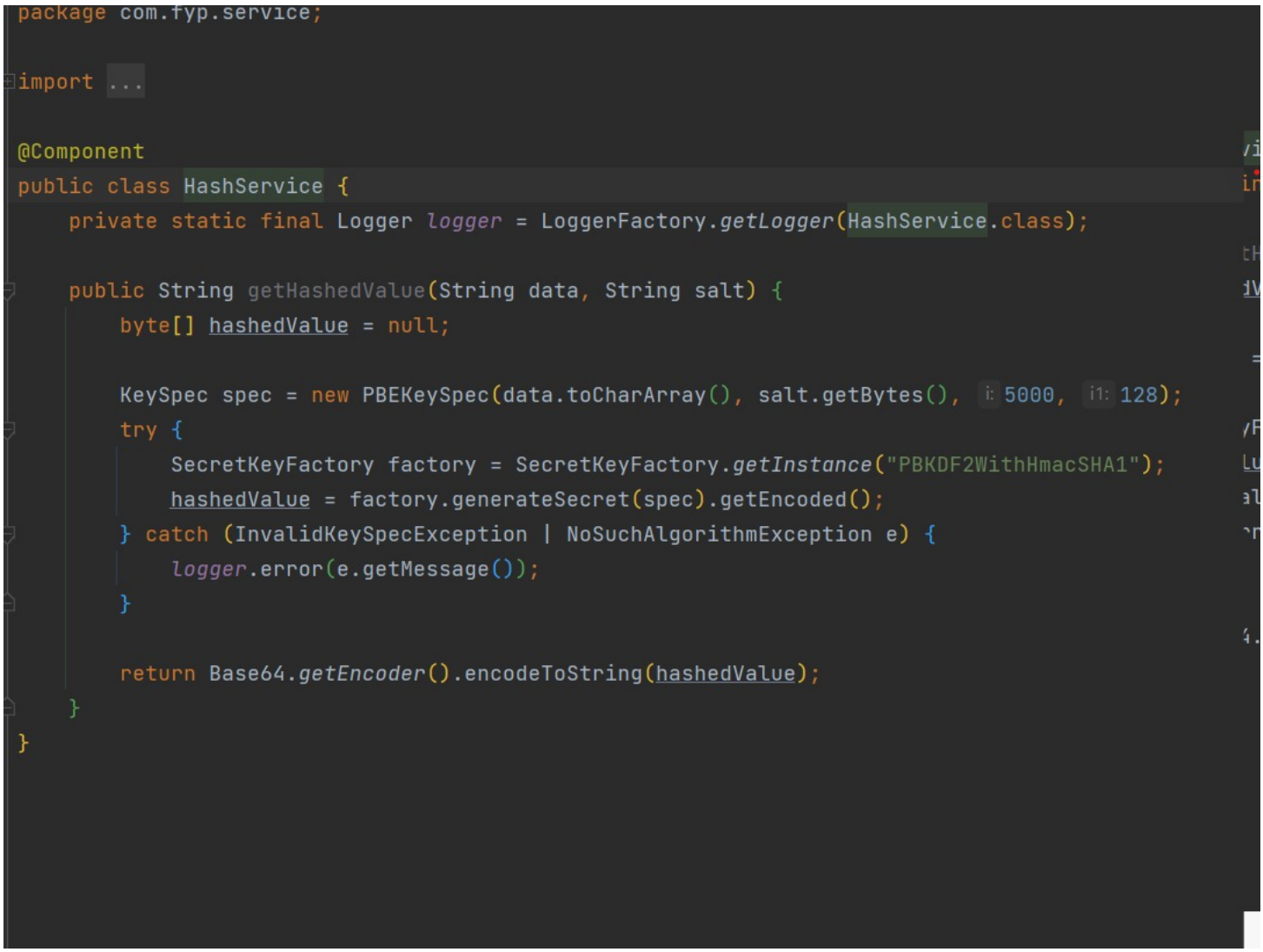
```
public String decryptValue(String data, String key) {
    byte[] decryptedValue = null;

    try {
        Cipher cipher = Cipher.getInstance("AES/ECB/PKCS5Padding");
        SecretKey secretKey = new SecretKeySpec(key.getBytes(), s: "AES");
        cipher.init(Cipher.DECRYPT_MODE, secretKey);
        decryptedValue = cipher.doFinal(Base64.getDecoder().decode(data));
    } catch (NoSuchAlgorithmException | NoSuchPaddingException
            | InvalidKeyException | IllegalBlockSizeException | BadPaddingException e) {
        logger.error(e.getMessage());
    }

    return new String(decryptedValue);
}

public static String getKey(){
    byte[] bytes = new byte[SEED];
    SecureRandom random = new SecureRandom();
    random.nextBytes(bytes);
   return Base64.getEncoder().encodeToString(bytes);
}
```

Figure 5.15: Method to dencrypt files in Encryption Service



```
package com.fyp.service;

import ...

@Component
public class HashService {
    private static final Logger logger = LoggerFactory.getLogger(HashService.class);

    public String getHashedValue(String data, String salt) {
        byte[] hashedValue = null;

        KeySpec spec = new PBEKeySpec(data.toCharArray(), salt.getBytes(), i: 5000, i1: 128);
        try {
            SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
            hashedValue = factory.generateSecret(spec).getEncoded();
        } catch (InvalidKeySpecException | NoSuchAlgorithmException e) {
            logger.error(e.getMessage());
        }

        return Base64.getEncoder().encodeToString(hashedValue);
    }
}
```

Figure 5.16: Hash Service to Hash the user's passwords

5.3.7 File Service

```java
package com.fyp.service;

import com.fyp.payload.FileDTO;
import org.springframework.web.multipart.MultipartFile;

import java.util.List;

public interface FileService {
    FileDTO getFileById(Long id);
    FileDTO insertFile(MultipartFile file,Long userId);
    FileDTO updateFile(Long id,FileDTO file);
    void deleteFile(FileDTO file);
    FileDTO getFileByName(String name);
    List<FileDTO> getAllFiles();
    List<FileDTO> getAllFilesByUserId(Long id);
}
```

Figure 5.17: File service interface which contains methods to be implemented with business logic

```java
public FileDTO insertFile(MultipartFile file,Long userId) {

    if (file == null) {
        throw new NullPointerException("Please Select a file to upload!");
    }
    if (file.getOriginalFilename() == null || file.getOriginalFilename().equals("")) {
        throw new NullPointerException("Please Select a file to upload!");
    }

    String fileName = StringUtils.cleanPath(file.getOriginalFilename());

    if (isExistingFile(fileName)) {
        throw new IllegalArgumentException("user-error: that file already exists | name conflicts with an existing f
    }

    FileDTO fileToSave = new FileDTO();
    fileToSave.setFileName(fileName);
    fileToSave.setContentType(file.getContentType());
    fileToSave.setFileSize((file.getSize()));
    fileToSave.setUserId(userId);
    try {
        fileToSave.setFileData(file.getBytes());
    } catch (IOException e) {
        e.printStackTrace();
    }
    fileToSave.setCreatedAt(Timestamp.valueOf(LocalDateTime.now()));
    return modelToDto(fileRepository.save(dtoToModel(fileToSave)));
}
```

Figure 5.18: File service interface which contains methods to be implemented with business logic

5.3.8 File Model

```java
@Entity
public class File {

    @Id
    @GeneratedValue(strategy =  GenerationType.IDENTITY)
    private Long fileId;
    private String fileName;
    private String contentType;
    private Long fileSize;
    private Long userId;
    private byte[] fileData;
    private Timestamp createdAt;
```

Figure 5.19: Entity class to store information about the file

5.3.9 Upload File

```
public FileDTO insertFile(MultipartFile file,Long userId) {

    if (file == null) {
        throw new NullPointerException("Please Select a file to upload!");
    }
    if (file.getOriginalFilename() == null || file.getOriginalFilename().equals("")) {
        throw new NullPointerException("Please Select a file to upload!");
    }

    String fileName = StringUtils.cleanPath(file.getOriginalFilename());

    if (isExistingFile(fileName)) {
        throw new IllegalArgumentException("user-error: that file already exists | name conflicts with an existing f
    }

    FileDTO fileToSave = new FileDTO();
    fileToSave.setFileName(fileName);
    fileToSave.setContentType(file.getContentType());
    fileToSave.setFileSize((file.getSize()));
    fileToSave.setUserId(userId);
    try {
        fileToSave.setFileData(file.getBytes());
    } catch (IOException e) {
        e.printStackTrace();
    }
    fileToSave.setCreatedAt(Timestamp.valueOf(LocalDateTime.now()));
    return modelToDto(fileRepository.save(dtoToModel(fileToSave)));
}
```

Figure 5.20: business implementation to upload a new file

5.3.10 Storage controller

```
    return ResponseEntity.ok(fileResponse);
}

@GetMapping("/getAll/{userId}")
public ResponseEntity<List<FileDTO>> getAllFiles(@PathVariable("userId") Long userId) {
    return ResponseEntity.ok(fileService.getAllFilesByUserId(userId));
}

@DeleteMapping("/delete{id}")
public ResponseEntity<ApiResponse> deleteFile(@PathVariable("id") Long fileId) {
    FileDTO file = fileService.getFileById(fileId);
    fileService.deleteFile(file);
    return ResponseEntity.ok(new ApiResponse( message: "File Deleted Successfully!", success: true));
}

@GetMapping("/download/{id}")
public ResponseEntity<ByteArrayResource> downloadFile(@PathVariable("id") Long fileId) {
    FileDTO file = fileService.getFileById(fileId);

    return ResponseEntity.ok()
            .contentType(MediaType.parseMediaType(file.getContentType()))
            .contentLength(file.getFileSize())
            .header(HttpHeaders.CONTENT_DISPOSITION, ...headerValues: "attachment; filename=\"" + file.getFileName() + "\"")
            .body(new ByteArrayResource(file.getFileData()));
}
```

Figure 5.21: Storage controller to handle storage related request

```
import com.fyp.payload.FileDTO;
import com.fyp.service.FileService;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.core.io.ByteArrayResource;
import org.springframework.http.HttpHeaders;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;
import org.springframework.web.multipart.MultipartFile;

import java.util.List;

@RestController
@RequestMapping("/api/v1/storage")
public class StorageController {

    @Autowired
    FileService fileService;

    @PostMapping("/upload")
    public ResponseEntity<FileDTO> uploadFile(@RequestParam("userId") Long userId, @RequestParam("file") MultipartFile file) {
        return ResponseEntity.ok(fileService.insertFile(file,userId));
    }

    @GetMapping("/view/{id}")
    public ResponseEntity<FileDTO> viewFile(@PathVariable("id") Long fileId) {
        FileDTO fileResponse = fileService.getFileById(fileId);
```

Figure 5.22: Storage controller to handle storage related request

CHAPTER 6

TESTING

Testing is an important part of any development process, if the product is not fully tested before the launch then it will cause lots of issues in the future and to correct those issues would be costlier than doing the testing process and correcting the bugs before the final launch.

6.1 TESTING ON DIFFERENT DEVICES

While building the web application we used 3 different devices with different screen size but it may be possible that users may utilize any other device so it is important to test the application on some of the devices.

Following Table shows the testing performed on different devices.

Table 6.1: Testing on Different Browser

Platform	**Load time**	**Text Arrangement**	**Elements and Control Objects**	**Functionality**
Google Chrome	GOOD	GOOD	GOOD	GOOD
Internet Explorer	GOOD	GOOD	GOOD	GOOD
Opera	GOOD	GOOD	GOOD	GOOD
Brave	GOOD	GOOD	GOOD	GOOD
Safari	GOOD	GOOD	GOOD	GOOD

We found the results satisfactorily, there wasn't any issue while launching our web application or with the look and feel of using the application.

6.2 TESTING TYPES

6.2.1 Functional Testing

1. **Different Users Are Able To Login With Different JWT Tokens**

   When users login through the login page they are securely checked

by the database data and logged in accordingly.

2. **Correctly Displaying The Home Page** When user logs in the home page is being shown according to the type of the user and the permission he or she has.

3. **Uploading, Deleting, Opening and Downloading the Stored Files** When the user logs in he can easily upload a new file, delete and existing file, Preview a stored file or Download any stored file to their machine.

6.2.2 Non Functional Testing

We are not going to use all of the non-functional testing techniques in our project, but only a few of them were used, which are as follows:

1. **Load Testing** In our web application we have to check whether the data e.g. user records are successfully coming from the MySQL database correctly or not, so we tested with the different internet connections of different bandwidth the data was coming smoothly in 2mb+ bandwidth of internet connection.

2. **Compatibility Testing** The GUI's are the most difficult part to manage when developing an web application, because there are many size screen computers are available in the market, so

we took average 3 screen sizes which are mostly available and we run our web application in all these 3 devices the design was perfectly fine and working smoothly.

3. **Black Box Testing** Black box testing is a type of test that only checks the functionality of the product and not the code or internal functioning of the project. In relation to our web application, black box tests have also been carried out to check the overall functioning of the website instead of the working or code structure. In this one we only gave a few inputs and checked the outputs and nothing else for the application.

   In Functional Testing we checked the basic functionalities of our web application discussed below:

   - Open the web application without any crash.
   - When user first logs in he sees all of the files that he has uploaded to the cloud.
   - User can choose any file to preview or perform actions like rename, delete and download.
   - When clicking the file a popup of that file shows a preview.
   - Clicking the download button fetches the file from the re-

mote server and stores it on the user's machine.

- If the user chooses to rename or delete a file the change is reflected immediately and updated in the database on the remote server.

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

In conclusion, cloud computing is here to stay for a long time and possibly forever unless something replaces it which seems unlikely at this point. Businesses and enterprises need to adopt the cloud technology and grow with it. The technology is both powerful and inspiring. In the long run, it proves to be a cost-effective way of executing services for many businesses, both big and small.

7.2 FUTURE WORK

Our application is an MVP (minimum viable product) of the final product which will have a lot more features with our main focus on securing sensitive data for large enterprises. Securing important Seismic data and encrypting and storing multi dimensional files to broaden the scope of the project and potentially make it successful in the market and eventually be considered the go-to software for cloud storage service.

CHAPTER 8

REFERENCES

[1] Tajammul, M., Parveen, R. Auto encryption algorithm for uploading data on cloud storage. Int. j. inf. tecnol. 12, 831–837 (2020). https://doi.org/10.1007/s41870-020-00441-9

[2] J. -J. Hwang, H. -K. Chuang, Y. -C. Hsu and C. -H. Wu, "A Business Model for Cloud Computing Based on a Separate Encryption and Decryption Service," 2011 International Conference on Information Science and Applications, 2011, pp. 1-7, doi: 10.1109/ICISA.

[3] F. Bracci, A. Corradi and L. Foschini, "Database security management for healthcare SaaS in the Amazon AWS Cloud," 2012 IEEE Symposium on Computers and Communications (ISCC), 2012, pp. 000812-000819, doi: 10.1109/ISCC.2012.6249401.

[4] Rafique, A., Van Landuyt, D. Joosen, W. PERSIST: Policy-Based Data Management Middleware for Multi-Tenant SaaS Leveraging Federated Cloud Storage. J Grid Computing 16, 165–194 (2018). https://doi.org/10.1007/s10723-018-9434-6

[5] Pancholi, Vishal R., and Bhadresh P. Patel. "Enhancement of

cloud computing security with secure data storage using AES." International Journal for Innovative Research in Science and Technology 2.9 (2016): 18-21.

[6] Kulkarni, G., Sutar, R., Gambhir, J. (2018). Cloud computing-Storage as service. International Journal of Engineering Research and Applications (IJERA), ISSN, 2248-9622.

[7] Spoorthy, V., M. Mamatha, and B. Santhosh Kumar. "A survey on data storage and security in cloud computing." International Journal of Computer Science and Mobile Computing 3, no. 6 (2019): 306-313.

[8] Pancholi, V. R., Patel, B. P. (2017). Enhancement of cloud computing security with secure data storage using AES. International Journal for Innovative Research in Science and Technology, 2(9), 18-21.

[9] Rafique, Ansar, et al. "Towards scalable and dynamic data encryption for multi-tenant saas." Proceedings of the Symposium on Applied Computing. 2017.

[10] Bracci, F., Corradi, A., Foschini, L. (2019, July). Database security management for healthcare SaaS in the Amazon AWS Cloud. In 2012 IEEE Symposium on Computers and Communications (ISCC)

(pp. 000812-000819). IEEE.

[11] Rafique, A., Van Landuyt, D., Reniers, V., Joosen, W. (2017, August). Leveraging NoSQL for scalable and dynamic data encryption in multi-tenant SaaS. In 2017 IEEE Trustcom/BigDataSE/ICESS (pp. 885-892). IEEE.

[12] Kartit, Z., Azougaghe, A., Kamal Idrissi, H., Marraki, M. E., Hedabou, M., Belkasmi, M., Kartit, A. (2019, September). Applying encryption algorithm for data security in cloud storage. In International Symposium on Ubiquitous Networking (pp. 141-154). Springer, Singapore.

[13] Yang, P., Xiong, N., Ren, J. (2020). Data security and privacy protection for cloud storage: A survey. IEEE Access, 8, 131723-131740.

[14] Liu, C., Zhu, L., Li, L. (2018, September). Fuzzy keyword search on encrypted cloud storage data with small index. In 2011 IEEE International Conference on Cloud Computing and Intelligence Systems (pp. 269-273). IEEE.

[15] Babitha, M. P., Babu, K. R. (2016, September). Secure cloud storage using AES encryption. In 2016 International Conference on Automatic Control and Dynamic Optimization Techniques (ICAC-

DOT) (pp. 859-864). IEEE.

[16] Deng, Z., Li, K., Li, K., Zhou, J. (2017). A multi-user searchable encryption scheme with keyword authorization in a cloud storage. Future Generation Computer Systems, 72, 208-218.

[17] Joshi, M., Moudgil, Y. S. (2019). Secure cloud storage. International Journal of Computer Science Communication Networks, 1(2), 171-175.

[18] Shen, W., Qin, J., Yu, J., Hao, R., Hu, J., Ma, J. (2019). Data integrity auditing without private key storage for secure cloud storage. IEEE Transactions on Cloud Computing, 9(4), 1408-1421.

[19] Shen, W., Qin, J., Yu, J., Hao, R., Hu, J. (2018). Enabling identity-based integrity auditing and data sharing with sensitive information hiding for secure cloud storage. IEEE Transactions on Information Forensics and Security, 14(2), 331-346.

[20] Bokefode, J. D., Bhise, A. S., Satarkar, P. A., Modani, D. G. (2016). Developing a secure cloud storage system for storing IoT data by applying role based encryption. Procedia Computer Science, 89, 43-50.

[21] Zeng, P., Choo, K. K. R. (2018). A new kind of conditional

proxy re-encryption for secure cloud storage. IEEE Access, 6, 70017-70024.